Technical Report: Lower-Bound Distance Queries under Partial Information

ACM Reference Format:
. 2026. Technical Report: Lower-Bound Distance Queries under Partial Information. In *Conference Name (Conference Acronym '26), Month DD–DD, 2026, City, Country*. ACM, New York, NY, USA, 4 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 COMPARISON WITH SHORTEST PATH (SP) BASED LANDMARKS [4]

: Qiao et al [4] propose SP based landmarks, which are a collection of nodes in the graph that provide upper bound on the SP distance between the given object-pair (o_i, o_j). The idea that the paper explores is to use the SP information from the landmark node to o_i (and o_j respectively), to "*stitch*" the paths. It provides better upper bound that combining the two shortest paths from o_i to the landmark and o_j to the landmark respectively. One could define the lower bound counterpart for the same. But, the approach does not provide a formal approach of selecting the landmarks for upper bounds and thus, does not provide any guarantees for lower bounds. On the flip side, we prove that our approach provides guarantees for the selection of "*edge-landmarks*".

To further illustrate the difference, assume one simple extension of Qiao et. al. [4] which selects SP-trees and applies the idea of stitching the paths "*efficiently*". To illustrate this consider Figure 1a, where (o_2, o_4) are selected are edge landmarks and correspondingly $\{o_2, o_4\}$ are selected as two SP based landmarks. The SP-trees for o_2 and o_4 are shown in Figures 1b and 1c respectively. The *stitched* path in the SP-tree between the query edge (o_5, o_7) is marked with a double edge in both these Figures. The *stitched* path is computed by finding the lowest common ancestor and then computing the path-length and the longest path in the stitched-path. A discussion on computing these two entities is presented later in the paper. Applying the edge-folding operation on these paths gives us a lower bound of 0, which is same as the trivial lower bound. Later, we illustrate the application of edge landmarks which provides a non-trivial LB by selecting the edge (o_2, o_4) as the edge landmark.

The two main drawbacks of this extension of Qiao et. al [4] is that (i) there is no good strategy/approach to select the landmarks, and (ii) the paper only discusses stitching the paths to find the path length, naively finding the longest edge adds a $O(n)$ in the worst case to the query time.

2 NP-HARDNESS PROOF

We provide a detailed proof to the NP-complete theorem below.

Theorem 1. k-EL problem is $\mathcal{NP}$-complete

Proof. We first prove that the verification is in P. A certificate for `Graph Egde Landmarks` problem consists of a graph, a set of k edges and a number representing the sum of lower bound values for the edges. Given a certificate for the `Graph Egde Landmarks` problem, we can verify whether the lower bound calculated using the given edges matches the provided/claimed number in $O((\binom{n}{2} - m) \cdot k)$ time .

We prove the NP-completeness by constructing an instance of a graph from `Set Cover` Problem . The input to the `Set Cover` Problem is a universe $\mathcal{U}$ of elements and a set of sets S whose union forms the universe. As an illustration, let us assume the following instance of `Set Cover` Problem , $\mathcal{U} = \{1, 2, 3, 4, 5\}$ and $\mathcal{S} = \{\{1, 3, 4\}, \{2, 3, 4\}, \{1, 4, 5\}\}$.

In this construction, we create four layers of vertices, namely A, B, C, D. Edges exist between two consecutive layers only, *i.e.*, A and B, B and C, and C and D. We now describe our construction.

For every element $e \in \mathcal{U}$, a vertex is created in the layer D of the graph. For every set $s_i \in \mathcal{S}$, we create two vertices B_i and C_i in layers B and C, respectively. Additionally, B_i is connected to C_i by an edge of length 3. For each element $e \in s_i$, we connect C_i to the node that corresponds to e with an edge of length 1.

Finally, we add $p > k(m + 2n - 3) - 2n$ vertices (A_1 to A_p) to the layer A in the graph. We will later provide the intuition behind the choice of p. Every vertex v in the layer A is connected to every vertex B_i in the layer B by a unit length edge. The constructed graph for the sample example before is presented in Figure 2.

The key idea behind this construction is the following, *the sum of the lower bound values after selecting an edge landmark set of size k which corresponds to a valid set cover of size k can be distinguished from the case when the landmark-set does not correspond to a set cover.* To further elaborate, whenever a set cover of size k exists, selecting the corresponding edges $B_i - C_i$ as the k-edge landmarks gives us the tightest lower bound values for all pairs of vertices between the layers A and D. Thus, we should select p such that we can distinguish between two cases.

As the edges in this graph that exist between layers B and C are of length 3 and the rest of the edges in the graph are of length 1, the $B - C$ edges need to be a part of the edge-folding operation to obtain a non-zero lower bound. Based on this observation, we note down the lower bound values between the different layers of the graph in Table 1.

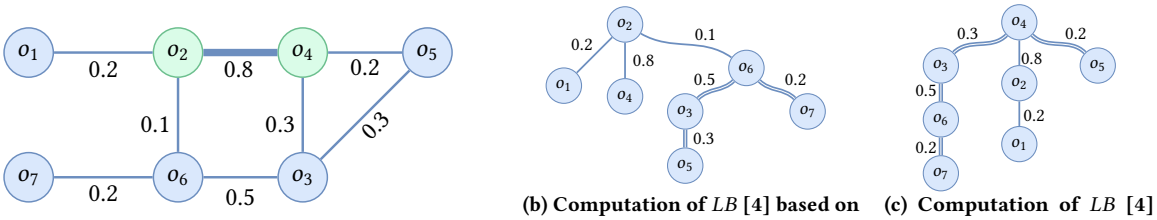
The row **Actual** represents the exact lower bound values, **K-EL** represents the lower bound values for the case when the k-edge landmarks correspont to a set cover. The row **K-Other** represents the values for the case when k edges (from the $B - C$ layer) are chosen as landmarks.

We now describe the values in the Table and then reason about the choice of p.

- The entries for $A - C$ only depends on the number of landmarks chosen and hence both `K-EL` and `K-Other` have a value of $2kp$.

Conference Acronym '26, Month DD–DD, 2026, City, Country

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX



(a) Edge landmark $\{(o_2, o_4)\}$ in Toy Example.

(b) Computation of LB [4] based on SP-Tree at o_2

(c) Computation of LB [4] based on SP-Tree at o_4

Figure 1: Illustration of Qaio et. al. [4] on edge (o_5, o_7), with SP-trees from o_2 and o_4 in Toy Example.

Cost \ Pair	$A-C$	$A-D$	$B-C$	$B-D$
Actual	$2pm$	pn	$m(m-1)$	$2\sum_{i\leq m} d_{C_i} - 1$
K-EL	$2kp$	pn	$> k(m-1)$	$2\sum_{i\in set} d_{C_i} - 1 \geq 2n$
K-Other	$2kp$	$\leq p(n-1)$	$\leq 2k(m-1)$	$2\sum_{i\in set} d_{C_i} - 1 < 2k(n-1)$

Table 1: ℓ_1 cost table for ideal, `K-EL`, and `K-Other` cases.

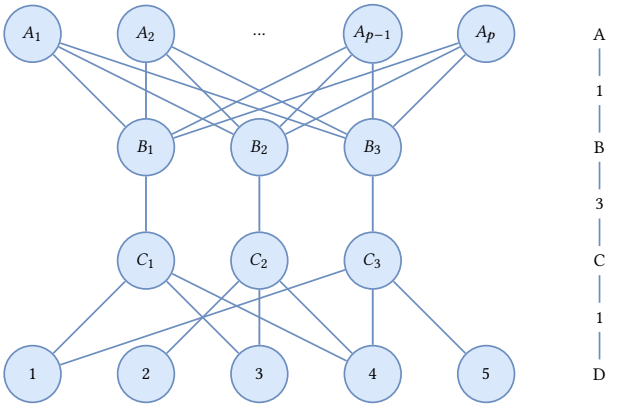


Figure 2: Graph Construction for the proof of `ELM` NP Completeness

- The value for the missing edges in $A-D$ depends on the quality of the landmarks. If there exists a `K-EL` that corresponds to a set cover then every vertex in D is connected to every vertex in A and hence will have a value of pn. But, if it is not a set cover, then the sum of lower bound values must at most be $p(n-1)$.
- Applying similar logic to $B-D$ layer, the sum of lower bound values of `K-EL` is at least equal to the $2\sum d_{C_i} - 1$, where d_{C_i} is the degree of the node C_i. But, for `K-Other` case, at least one of the edge between $B-D$ has a lower bound of 0. Hence, the sum of lower bound values cannot exceed $2k(n-1)$.
- There are three cases for the edges in $B-C$ layer. If the edge-landmark contains $B_i - C_i$, then every missing edge from C_i to B_j has a tight lower bound value of 1. Hence, irrespective of the choice of landmarks $k(m-1)$ missing edges will have a lower bound value of 1. In the second case, if a pair of vertices $B_a - C_b$ are chosen such that neither $B_a - C_a$ nor $B_b - C_b$ are part of the edge-landmarks set, then the lower bound will be 0. In the third case, if edge-landmack contains $B_i - C_i$, then for a missing edge B_i to C_j where C_j is not part of the k-EL then there may be a path from $C_j - D - C_i - B_i$ where there is a vertex in the layer D which has an edge to both C_j and C_i. Such a path may exist for a large number of `k-Other` and hence they may contribute to at most $k(m-1)$. But we consider the worst case scenario for the `k-EL`.

The ℓ_1 sum of the lower bound values for `k-EL` and `k-Other` can be distingushable if the sum of `k-EL` is greater than that of `k-Other`. The calculations below show that such a distinction is possible.

$$2kp + pn + k(m-1) + 2n > 2kp + p(n-1) + 2k(m-1) + 2k(n-1)$$

Simplifying the above we get,

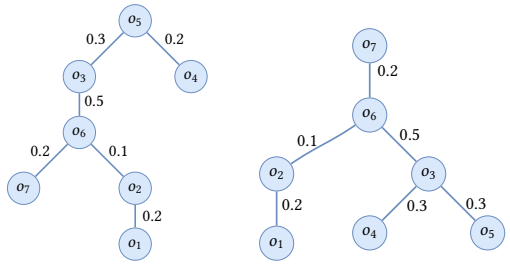
$$p > k(m + 2n - 3) - 2n$$

Hence, proved. □

3 DETAILS ABOUT CONTRIBUTION OF AN EDGE — `SGEL` USING SP-TREE

The sampling based greedy algorithm, at each step, relies on computing the LB contribution using a sampled edge $e \in E - E'$. With the modification of the function LB in Equation **??**, our assessment function needs to be updated to compute the LB score "*quickly*" (with Equation **??** alone computation consumed $O(1)$ time per edge). At each step in the algorithm, SP-tree is computed from o_i and o_j for a sampled edge (o_i, o_j). Our goal is to utilize only this information to compute contributions quickly. We first propose the naive approach to compute the contribution.

Naive approach: The tree based constraints as part of the optimization function as shown in Equation **??** rely on computing (i) `maximum` and (ii) `sum` of the set $\mathcal{VV}_{o_x}$ (and $\mathcal{VV}_{o_y}$) for each edge $(o_x, o_y) \in E'$ quickly. Given the unknown edge (o_i, o_j), SP is computed from o_i and o_j as a first step in the computation of LB.



(a) SP-tree from o_5 **(b) SP-tree from o_7**

Figure 3: Using shortest path tree of the two end points to compute the effect on LB.

One naive method to approach the problem is to maintain "*jump pointers*", following the idea of "*binary lifting*" [1, 2]. The idea proposed by Bender et al. [1] is to maintain jump pointers. The jump pointers are maintained for $\forall_{i\in[\,\lfloor d\rfloor\,]}\ 2^i$th ancestor from every vertex in the tree, where d is the depth of the node in the tree. Each shortest-path tree τ_{o_i} and τ_{o_j} is augmented so that every node o_z maintains: (i) its root-to-node distance, and (ii) the maximum-weight edge on the corresponding root-to-o_z path.

Consider an iteration of SGEL where the edge (o_i, o_j) is sampled. We first construct the SP trees rooted at o_i and o_j. To evaluate the additional contribution of a candidate edge $(o_x, o_y) \in E'$, our goal is to locate "*shallowest*" common ancestor of o_x in τ_{o_i} and τ_{o_j}. We repeat the same process for o_y.

Let d_i and d_j denote the depths of o_x in τ_{o_i} and τ_{o_j}, respectively. Our goal is to find the lowest node o_a that lies on the path from o_x to the root in both trees. This node is precisely $\mathtt{LCA}_{o_x}(o_i, o_j)$, and computing it efficiently is the key step.

We maintain two pointers p_i and p_j initialized to o_x in τ_{o_i} and τ_{o_j}. Using binary lifting, we start our search for the shallowest common ancestor using jump pointers, starting from $2^{\lfloor \log_2(\min(d_i,d_j))\rfloor}$. If the two jump pointers are pointing to the same node, we need to explore the upper part of the tree and we restart the search from that point onward. If found to be different, we go the jump pointer of a level lower and continue this process. Once the shallowest common ancestor is found, the length of the path and the maximum edge-weight along the path from this ancestor to the root can be retrieved to compute the lower bound.

This procedure takes $O(\log(n))$ time per edge due to the binary searches in the two trees, and requires $O(n\log(n))$ time and space to preprocess each tree. Over sk sampled edges, the total preprocessing time becomes $O(sk(m+n)\log(n)) = O(skm\log(n))$.

Instead of this expensive *naive* approach, we propose an approach that computes both the contribution in quickly ($O(1)$ time) in linear space for every SP-tree.

A key observation simplifies the computation. The edges on the path from o_x to $\mathtt{LCA}_{o_x}(o_i, o_j)$ in the SP-tree rooted at o_x are exactly the same as the edges on the path from o_i to that same LCA in the SP-tree rooted at o_i.

This is illustrated in Figure 3. Consider the SP-trees rooted at o_5 and o_7. Suppose we want to evaluate the effect of adding an edge involving o_2. From the SP-tree rooted at o_2 (Figure **??**), we see that $\mathtt{LCA}_{o_2}(o_5, o_7) = o_6$. The path-edge set $\mathcal{VR}_{o_2}(o_6) = \{(o_2, o_6)\}$, and this is exactly the same edge that appears on the path from o_2 to o_6 in the SP-tree rooted at o_5.

Formally,

$$\mathcal{VV}_{o_x}\big(o_i, \mathtt{LCA}_{o_x}(o_i, o_j)\big) = \mathcal{VR}_{o_i}\big(\mathtt{LCA}_{o_x}(o_i, o_j)\big).$$

Using this, we rewrite the tree constraint in Equation **??** as

$$\mathcal{VV}_{o_x}(o_i, o_j) = \mathcal{VR}_{o_i}\big(\mathtt{LCA}_{o_x}(o_i, o_j)\big) \ \cup\ \mathcal{VR}_{o_j}\big(\mathtt{LCA}_{o_x}(o_i, o_j)\big).$$

Since the sum and maximum over vertex-root path sets can be computed in $O(n)$ time across all vertices, the main remaining task is to compute $\mathtt{LCA}_{o_x}(o_i, o_j)$ efficiently.

We compute LCAs using a memoized upward walk on the two SP trees starting from o_x. For a given vertex o_x, let t_{o_i} and t_{o_j} denote its nodes in τ_{o_i} and τ_{o_j}, respectively. We advance both pointers to their parents in lockstep while the current nodes remain identical, collecting visited nodes into a temporary list $\mathcal{V}$. A lookup table $\mathcal{H}$ is maintained for already computed results.

If we encounter a node already present in the lookup table $\mathcal{H}$, we set $\mathcal{H}[u] = \mathcal{H}[t]$ for every $u \in \mathcal{V}$ (where t is the encountered node) and return the stored LCA. If the two pointers diverge, the last matching node is the LCA; we store this LCA in $\mathcal{H}$ for all nodes in $\mathcal{V}$ and return it. Because each vertex is added to the table $\mathcal{H}$ at most once, filling $\mathcal{H}$ for all n vertices requires $O(n)$ time and $O(n)$ space. We present Reversed LCA in Algorithm 1.

Algorithm 1: Memoized Reverse $\mathtt{LCA}_{o_x}(o_i, o_j)$

Input: Vertex o_x in τ_{o_i}, τ_{o_j}; table $\mathcal{H}$
Output: $\mathtt{LCA}_{o_x}(o_i, o_j)$

$t_i \leftarrow$ node of o_x in τ_{o_i};
$t_j \leftarrow$ node of o_x in τ_{o_j};
$\mathcal{V} \leftarrow \emptyset$;
while $\mathtt{parent}(t_i) = \mathtt{parent}(t_j)$ **do**
  $\mathcal{V} \leftarrow \mathcal{V} \cup \{t_i\}$;
  if $t_i \in \mathcal{H}$ **then**
    foreach $u \in \mathcal{V}$ **do**
      $\mathcal{H}[u] \leftarrow \mathcal{H}[t_i]$;
    return $\mathcal{H}[t_i]$;
  $t_i \leftarrow \mathtt{parent}(t_i)$;
  $t_j \leftarrow \mathtt{parent}(t_j)$;
foreach $u \in \mathcal{V}$ **do**
  $\mathcal{H}[u] \leftarrow t_i$;
return t_i;

4 MISSING PROOFS

Theorem 2. The goal function F for k-EL problem (Equation **??**) is monotone and submodular.

Proof. To prove the monotone property of the function F, let us add a new edge e to the set L to see its impact on F.

$$F(L \cup \{e\}) = \sum_{i=1}^{n}\sum_{j=i+1}^{n} LB((o_i, o_j), L \cup \{e\})$$

As the maximum in the LB function is calculated over an additional edge (e), the function LB must return a larger value when provided

with the additional edge e. This can be written as,

$$LB((o_i, o_j), L \cup \{e\}) \geq LB((o_i, o_j), L)$$

Summing the above equation over all object-pairs (o_i, o_j)), we have the monotone property $F(L \cup \{e\}) \geq F(L)$.

To prove the sub-modularity property, let us consider two sets of landmarks $L_1 \subset L_2$. Let us compute the gain in F when an additional edge is added to the two sets of landmarks L_1 and L_2.

Using the monotone property for any edge (o_i, o_j), we know that $LB((o_i, o_j), L_1) \geq LB((o_i, o_j), L_2)$. But, $LB((o_i, o_j), \{e\})$ could either be (i) greater than both terms, (ii) greater than the L_2 term, or (iii) smaller than both terms. If an edge belongs to case (iii), the contribution is 0 to both terms. But in cases (i) and (ii) the contribution of edge $\{e\}$ is greater to L_2 than L_1. Hence, the sub-modular property holds.

□

Theorem 3. The time taken by GEL to select the k-edge landmarks is $O(kn^2m)$.

Proof. The greedy algorithm consumes $O(1)$ time to compute the lower bound score for one of the object-pairs using a single edge $e \in E$. As there are n^2 object-pairs, computing each contribution takes n^2 time per edge. Over the set of edges E which has size m, the total time consumed is $O(n^2m)$. Additionally, as there are k iterations, the total time consumed is $O(kn^2m)$.

□

Theorem 4. The optimization function given in Equation **??**, with Equation **??** as the function for LB, is monotone and submodular.

Proof. This proof is similar to the proof of Theorem 2. To prove the monotone property of the function F, let us add a new edge e to the set L to see its impact on F. As the maximum in the LB function is calculated over an additional edge (e) and the SP-Tree, the function LB must return a larger value when provided with the additional edge e. This can only improve the LB value and hence, the function is monotone.

The proof of sub-modularity is also very similar to the one from Theorem 2. To prove the sub-modularity property, let us consider two sets of landmarks $L_1 \subset L_2$. Let us compute the gain in F when an additional edge is added to the two sets of landmarks L_1 and L_2.

Using the monotone property for any edge (o_i, o_j), we know that $LB((o_i, o_j), L_1) \geq LB((o_i, o_j), L_2)$. But, $LB((o_i, o_j), \{e\})$ could either be (i) greater than both terms, (ii) greater than the L_2 term, or (iii) smaller than both terms. If an edge belongs to case (iii), the contribution is 0 to both terms. But in cases (i) and (ii) the contribution of edge $\{e\}$ is greater to L_2 than L_1. Hence, the sub-modular property holds.

□

Theorem 5. Given a metric space graph $G(V, E)$ and an edge e, the number of object-pair samples s required to obtain an estimate on the sum of lower bounds between all pairs of objects within an error of ϵ and a confidence of $1 - \delta$ is $s \geq \frac{c^2 \ln(2/\delta)}{2\epsilon^2}$, where c is the diameter of the graph.

Proof. Let s be the number of samples used for the lower bound estimation. Consider random variables $X_1, X_2, \cdots X_s$ corresponding to the s samples. As we want to estimate the probability of the tail varying more than δ, we use Hoeffding's inequality [3]. For this derivation, let $S = \sum_{i=1}^{s} X_i$ and $\mathbb{E}[S] = \sum_{i=1}^{s} \mathbb{E}[X_i] = s\mu$ where μ is expectation of a single sample.

$$P(|S - \mathbb{E}[S]| \geq t) \leq 2\, exp(-\frac{2t^2}{\sum_{i=1}^{s}(b_i - a_i)^2})$$

where $a_i \leq X_i \leq b_i$. As each of our random variables lies in the range of 0 and c ($a_i = 0$ and $b_i = c$), where c is the diameter of the graph, we get,

$$P(|S/s - \mu| \geq t/s) \leq 2\, e^{\frac{-2t^2}{c^2 s}}$$

Using $\epsilon = \frac{t}{s}$, we get $P(|S/s - \mu| \geq \epsilon) \leq 2\, e^{-2s\epsilon^2/c^2} \leq \delta$. Solving for δ, we get, $s \geq \frac{c^2 \ln(2/\delta)}{2\epsilon^2}$. Hence, proved.

□

Theorem 6. The expected time needed to calculate the k edge-landmarks is $n(1 - (1 - \frac{2}{n})^{sk})(m + n \log(n))$.

Proof. Consider the probability of a vertex being chosen in at least one of the sk iterations of our algorithm. For this proof, let us denote the probability as p.

$$p = 1 - \left(\binom{n-1}{2} / \binom{n}{2}\right)^{sk} = 1 - \left(1 - \frac{2}{n}\right)^{sk}$$

Using linearity of expectation, the total number of vertices from which shortest path needs to be computed is np which is given by $n(1 - (1 - \frac{2}{n})^{sk})$. Hence, the expected amount of time needed to compute the k-edge landmarks is $n(1 - (1 - \frac{2}{n})^{sk}) \cdot (m + n \log n)$. Additionally, the number of shortest path tends to $2sk$ as n tends to ∞, *i.e.* $lim_{n\to\infty}\, n(1 - (1 - \frac{2}{n})^{sk}) = 2sk$, which corroborates our worst case analysis above. Hence, proved.

□

REFERENCES

[1] Michael A Bender and Martın Farach-Colton. 2004. The level ancestor problem simplified. *Theoretical Computer Science* 321, 1 (2004), 5–12.

[2] Michael A Bender, Martin Farach-Colton, Giridhar Pemmasani, Steven Skiena, and Pavel Sumazin. 2005. Lowest common ancestors in trees and directed acyclic graphs. *Journal of Algorithms* 57, 2 (2005), 75–94.

[3] Wassily Hoeffding. 1994. Probability inequalities for sums of bounded random variables. *The collected works of Wassily Hoeffding* (1994), 409–426.

[4] Miao Qiao, Hong Cheng, Lijun Chang, and Jeffrey Xu Yu. 2012. Approximate shortest distance computing: A query-dependent local landmark scheme. *IEEE Transactions on Knowledge and Data Engineering* 26, 1 (2012), 55–68.